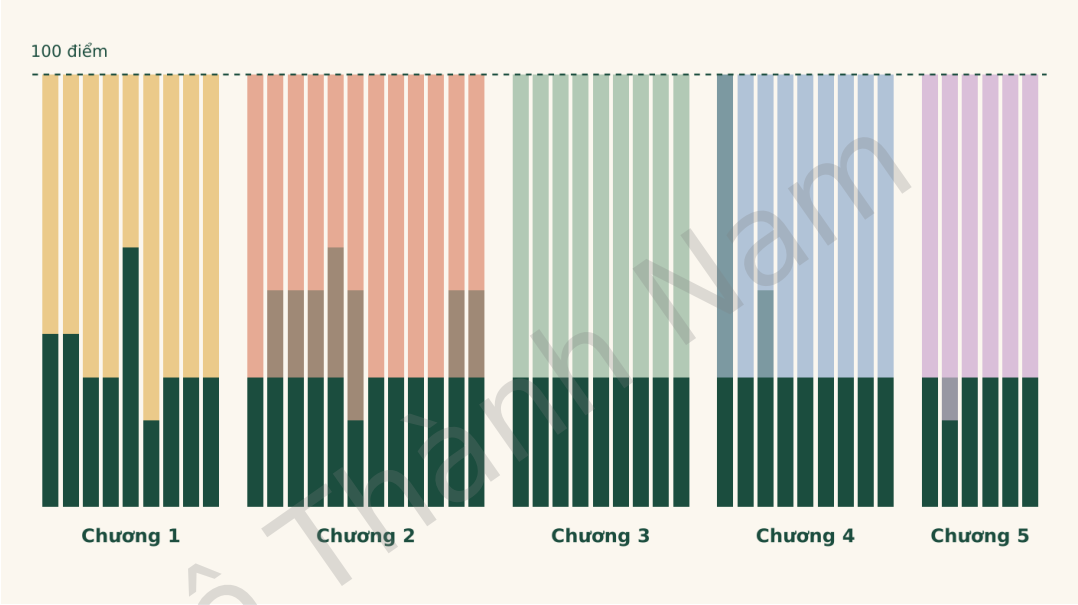
Tổng hợp và biên soạn: Lê Thành Nam

TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

CENTROID

TRỌNG TÂM CÂY · PHÂN RÃ TRỌNG TÂM

TẬP 3 *Hướng dẫn giải chi tiết*



Mỗi cột là một bài: phần sẫm là điểm của cách ngây thơ, cả cột là lời giải tối ưu.

Lê Thành Nam

45	109	630
LỜI GIẢI	CHƯƠNG TRÌNH C++	TEST CHẤM THẬT

01 **TRỌNG TÂM CỦA CÂY** · 9 lời giải

02 **PHÂN RÃ TRỌNG TÂM VÀ ĐẾM ĐƯỜNG ĐI** · 12 lời giải

03 **CÂY TRỌNG TÂM VÀ TRUY VẤN** · 9 lời giải

04 **KĨ THUẬT KẾT HỢP CÁC NHÁNH** · 9 lời giải

05 **PHÂN RÃ TRỌNG TÂM NÂNG CAO** · 6 lời giải

LỜI NÓI ĐẦU

Tập 3 trình bày lời giải đầy đủ cho 45 bài tập của Tập 2. Mỗi lời giải không dừng ở chương trình đạt 100 điểm, mà đi lại đúng con đường một học sinh giỏi suy nghĩ trong phòng thi: bắt đầu từ cách làm đầu tiên ai cũng nghĩ ra (thường là BFS từ mọi đỉnh, hay xét mọi cặp đỉnh), đo xem nó được bao nhiêu điểm và vì sao không đi xa hơn, rồi phá bế tắc bằng một quan sát để đưa bài toán về trọng tâm hoặc phân rã trọng tâm.

Điều xuyên suốt cả tập sách: cách ngây thơ nhìn vào mọi cặp đỉnh, lời giải tối ưu chỉ nhìn vào các đường đi qua một trọng tâm, rồi để phân rã lo phần còn lại. Khi đọc từng lời giải, hãy tự trả lời ba câu hỏi của Tập 1: *Mỗi nửa đường mang thông tin gì, ghép hai nửa bằng gì, và loại cặp cùng nhánh thế nào?* Với các bài truy vấn, thêm câu hỏi thứ tư: *Mỗi trọng tâm cần giữ tóm tắt gì về mảnh của mình?*

Mọi điểm số trong mục Bế tắc và Bảng điểm là điểm chấm thật, không phải ước lượng:

Bộ test. Với mỗi bài, người biên soạn viết bộ sinh test theo đúng ràng buộc từng subtask. Mỗi subtask có cây ở nhiều hình dạng: ngẫu nhiên, đường thẳng, hình sao, con sâu, nhị phân, "cây sâu" (cha cách con không quá ba chỉ số), cây chổi và cây "căn bậc hai"; các đỉnh luôn được đánh số lại ngẫu nhiên. Tổng cộng 630 test cho 45 bài.

Đáp án chuẩn. Đáp án của mỗi test do lời giải mẫu sinh ra. Trước đó, mọi chương trình trong sách đã được đối chiếu với lời giải vét cạn viết độc lập bằng Python (của Tập 2) trên vài trăm cây ngẫu nhiên nhỏ cho mỗi bài; lời giải mẫu còn được đối chiếu với cách ngây thơ (chạy không giới hạn thời gian) trên các cây cỡ vài nghìn tới một vạn đỉnh, và các bài dùng FFT được kiểm tra với công thức đúng trên đường thẳng và hình sao lớn nhất.

Chấm. 109 chương trình (cách ngây thơ, lời giải trung gian và lời giải tối ưu) được biên dịch bằng g++ 13 với `-O2 -std=c++14` và chạy từng test với giới hạn thời gian của đề (1 hoặc 2 giây) và 256 MB. Một subtask chỉ được điểm khi mọi test trong subtask đều đúng. Bài CHỈ HUY cho phép nhiều đáp án nên được chấm bằng trình kiểm tra riêng; bài TRUNG BÌNH LỚN NHẤT chấp nhận sai số 10^{-3}.

Trên một máy chấm khác, thời gian chạy có thể chênh lệch đôi chút; máy dùng để đo khá chậm khi truy cập bộ nhớ ngẫu nhiên, nên số đo trong sách là "bi quan". Khoảng cách giữa các cách làm (thường hàng nghìn lần) thì không đổi.

Quá trình kiểm chứng cũng là quá trình học. Phiên bản đầu của nhiều lời giải dùng danh sách kề `vector<vector<int>>` và chạy gần 1 giây trên cây đường thẳng $2 \cdot 10^5$ đỉnh, chỉ vì mỗi danh sách kề nằm ở một chỗ khác nhau trong bộ nhớ. Chuyển sang danh sách kề phẳng (mục Khung chương trình) làm chương trình nhanh gấp hai đến ba lần mà không đổi một dòng thuật toán nào. Lời giải TỔNG CHIA HẾT và CÙNG MÀU, ĐÚNG K cũng được viết lại để bỏ bớt một lần sắp xếp. Những câu chuyện ấy được giữ lại trong sách vì chúng là kinh nghiệm thật của phòng thi.

Chúc các em học tập say mê và đạt kết quả cao!

Lê Trọng Nam

CẤU TRÚC MỘT LỜI GIẢI

Mục	Nội dung
1. Hiểu bài toán	Diễn đạt lại đề bằng ngôn ngữ của trọng tâm và đường đi qua trọng tâm; chỉ ra mô hình quen thuộc của Tập 1 ẩn sau câu chuyện. Thường kèm hình vẽ của ví dụ.
2. Cách ngây thơ	Ý tưởng, chương trình đầy đủ và độ phức tạp của cách làm đầu tiên ai cũng nghĩ ra.
3. Bế tắc	Điểm thực đo của cách ngây thơ trên từng subtask, kèm lí do mất điểm. Nhiều bài có thêm một lời giải trung gian: lời giải riêng cho subtask đặc biệt, hoặc một lời giải sai "trông như đúng". Kết thúc bằng hộp ĐIỂM BẾ TẮC: câu hỏi then chốt cần trả lời.
4. Phá bế tắc	Các bước của ý tưởng, bảng chạy tay trên ví dụ, và hộp VÌ SAO ĐÚNG?.
5. Cách tối ưu	Chương trình hoàn chỉnh (đã được kiểm chứng), độ phức tạp, và hướng mở rộng.
6. Bẫy thường gặp	Những lỗi làm mất điểm hay gặp nhất.
7. Bảng điểm	Điểm thực tế của từng cách làm trên bộ test. Dòng tô xanh là lời giải mẫu.

Mọi chương trình đều đọc ghi theo kiểu "có tệp thì đọc tệp": nếu tồn tại tệp `.INP` thì đọc từ tệp và ghi ra `.OUT`, ngược lại dùng bàn phím và màn hình. Không chương trình nào duyệt cây bằng đệ quy.

Để sách gọn, năm đoạn mã dùng chung (khung phân rã, cây trọng tâm, FFT) chỉ được in một lần ở mục KHUNG CHƯƠNG TRÌNH ngay sau đây. Trong các chương trình, mỗi đoạn được thay bằng một dòng chú thích, ví dụ `// [KHUNG A] ...`; khi chép chương trình, hãy chép nguyên văn đoạn khung tương ứng vào đúng vị trí đó. Các chương trình đã chấm là các chương trình đầy đủ như vậy.

> **LỜI KHUYÊN**
>
> Đừng đọc lời giải như đọc truyện. Sau mục 3, hãy gấp sách và tự tìm cách phá bế tắc trong ít nhất mười lăm phút. Câu hỏi trong hộp ĐIỂM BẾ TẮC chính là gợi ý mạnh nhất.
>
> Sau mục 5, hãy tự viết lại chương trình mà không nhìn sách, rồi tự sinh cây ngẫu nhiên nhỏ (nhớ có cả đường thẳng và hình sao) để đối chiếu với cách ngây thơ của chính em.

Lê Thành Nam

KHUNG CHƯƠNG TRÌNH

Các đoạn mã dưới đây được dùng chung bởi nhiều lời giải. Mỗi đoạn đều đã nằm trong các chương trình được chấm; ở đây chúng được in một lần, kèm giải thích.

KHUNG A. Danh sách kề phẳng và phân rã trọng tâm (cạnh không có độ dài)

Thay cho `vector<vector<int>> g`, các láng giềng của mọi đỉnh được cất liền nhau trong một mảng `to`: láng giềng của v là to[k] với beg[v] ≤ k < beg[v + 1]. Hàm `readTree` đọc n − 1 cạnh, đếm bậc, lấy tổng tiền tố để có `beg`, rồi rải các đầu mút vào `to`. Cách cất này giữ dữ liệu liền mạch trong bộ nhớ: trên cây đường thẳng $2 \cdot 10^5$ đỉnh với chỉ số xáo trộn, riêng phần phân rã nhanh hơn khoảng 2,5 lần so với `vector<vector<int>>`.

`collect(r)` thu thập mảnh chứa r theo thứ tự BFS (bỏ qua đỉnh đã xóa) và tính kích thước cây con theo gốc r. `centroid()` đi từ r về phía cây con có hơn m/2 đỉnh cho tới khi không còn cây con nào như vậy: đó là một trọng tâm (Bài 1, Tập 1). Phép đi này chỉ duyệt danh sách kề của các đỉnh trên đường đi, rẻ hơn việc kiểm tra mọi đỉnh.

Lê Thành Nam

KHUNG A: n, beg, to, par, sz, order, removed; readTree, collect, centroid

```
int n;
// láng giềng của v: to[k] với beg[v] ≤ k < beg[v + 1]
vector<int> beg, to;
vector<int> par, sz, order;
vector<char> removed;

// đọc n − 1 cạnh, dựng danh sách kề phẳng
void readTree() {
    vector<int> U(max(n - 1, 0)), V(max(n - 1, 0));
    beg.assign(n + 2, 0);
    for (int i = 0; i < n - 1; i++) { cin >> U[i] >> V[i]; beg[U[i] + 1]++;
beg[V[i] + 1]++; }
    for (int v = 1; v <= n + 1; v++) beg[v] += beg[v - 1];
    to.assign(2 * max(n - 1, 0), 0);
    vector<int> pos(beg);
    for (int i = 0; i < n - 1; i++) { to[pos[U[i]]++] = V[i]; to[pos[V[i]]++]
= U[i]; }
    par.assign(n + 1, 0); sz.assign(n + 1, 0); removed.assign(n + 1, 0);
}
// Thu thập mảnh chứa r (bỏ qua các đỉnh đã xóa) theo thứ tự BFS và tính kích
thước cây con
void collect(int r) {
    order.assign(1, r);
    par[r] = 0;
    for (size_t i = 0; i < order.size(); i++) {
        int v = order[i];
        sz[v] = 1;
        for (int k = beg[v]; k < beg[v + 1]; k++) {
            int c = to[k];
            if (!removed[c] && c != par[v]) { par[c] = v; order.push_back(c);
}
        }
    }
    for (int i = (int)order.size() - 1; i > 0; i--) sz[par[order[i]]] +=
sz[order[i]];
}
// Đi từ gốc mảnh về phía cây con có hơn m / 2 đỉnh; dừng ở đâu là trọng tâm
ở đó
int centroid() {
    int m = order.size(), v = order[0];
    while (true) {
        int nx = 0;
        for (int k = beg[v]; k < beg[v + 1]; k++) {
            int c = to[k];
            if (!removed[c] && c != par[v] && 2 * sz[c] > m) { nx = c; break;
}
        }
        if (!nx) return v;
        v = nx;
    }
}
```

Lê Thành Nam

KHUNG B. Như KHUNG A, cạnh có độ dài

Thêm mảng `wt` song song với `to`: cạnh thứ k (trong danh sách kề phẳng) có độ dài wt[k]. Các hàm `collect` và `centroid` giống hệt KHUNG A.

KHUNG B: thêm wt

```
int n;
// láng giềng của v: to[k], độ dài cạnh wt[k], beg[v] ≤ k < beg[v + 1]
vector<int> beg, to, wt;
vector<int> par, sz, order;
vector<char> removed;

// đọc n - 1 cạnh có độ dài, dựng danh sách kề phẳng
void readTree() {
    vector<int> U(max(n - 1, 0)), V(max(n - 1, 0)), W(max(n - 1, 0));
    beg.assign(n + 2, 0);
    for (int i = 0; i < n - 1; i++) { cin >> U[i] >> V[i] >> W[i]; beg[U[i] + 1]++; beg[V[i] + 1]++; }
    for (int v = 1; v <= n + 1; v++) beg[v] += beg[v - 1];
    to.assign(2 * max(n - 1, 0), 0); wt.assign(2 * max(n - 1, 0), 0);
    vector<int> pos(beg);
    for (int i = 0; i < n - 1; i++) {
        to[pos[U[i]]] = V[i]; wt[pos[U[i]]++] = W[i];
        to[pos[V[i]]] = U[i]; wt[pos[V[i]]++] = W[i];
    }
    par.assign(n + 1, 0); sz.assign(n + 1, 0); removed.assign(n + 1, 0);
}
// Thu thập mảnh chứa r (bỏ qua các đỉnh đã xóa) theo thứ tự BFS và tính kích thước cây con
void collect(int r) {
    order.assign(1, r);
    par[r] = 0;
    for (size_t i = 0; i < order.size(); i++) {
        int v = order[i];
        sz[v] = 1;
        for (int k = beg[v]; k < beg[v + 1]; k++) {
            int c = to[k];
            if (!removed[c] && c != par[v]) { par[c] = v; order.push_back(c); }
        }
    }
    for (int i = (int)order.size() - 1; i > 0; i--) sz[par[order[i]]] += sz[order[i]];
}
// Đi từ gốc mảnh về phía cây con có hơn m / 2 đỉnh; dừng ở đâu là trọng tâm ở đó
int centroid() {
    int m = order.size(), v = order[0];
    while (true) {
        int nx = 0;
        for (int k = beg[v]; k < beg[v + 1]; k++) {
            int c = to[k];
            if (!removed[c] && c != par[v] && 2 * sz[c] > m) { nx = c; break; }
        }
        if (!nx) return v;
        v = nx;
    }
}
```

Lê Thành Nam

KHUNG C. Cây trọng tâm: tổ tiên và khoảng cách, lưu phẳng

Tầng của cây trọng tâm không vượt quá $\lfloor \log_2 n \rfloor + 1 = 18$ khi $n \le 2 \cdot 10^5$. Thay cho `vector<vector<pair>>` như Tập 1, tổ tiên tầng t của v được cất ở ô $v \cdot 18 + t$ của một mảng phẳng: anc(v, t) là trọng tâm tổ tiên tầng t, dis(v, t) là khoảng cách từ v tới nó, lv[v] là tầng của chính v (nên tổ tiên của v là các tầng 0 … lv[v], và anc(v, lv[v]) = v). Trọng tâm con của tầng t chứa v chính là anc(v, t + 1).

KHUNG C: LG, ac, ad, lv, anc, dis, buildAnc (khoảng cách int)

```cpp
// số tầng tối đa của cây trọng tâm (2^18 > 2·10^5)
const int LG = 18;
// ac[v * LG + t]: trọng tâm tổ tiên tầng t của v; lv[v]: tầng của v
vector<int> ac, lv;
// ad[v * LG + t]: khoảng cách từ v tới tổ tiên đó
vector<int> ad;
// trọng tâm tổ tiên tầng t của v
int anc(int v, int t) { return ac[(size_t)v * LG + t]; }
// khoảng cách từ v tới nó
int dis(int v, int t) { return ad[(size_t)v * LG + t]; }
void buildAnc() {
    ac.assign((size_t)(n + 1) * LG, 0); ad.assign((size_t)(n + 1) * LG, 0);
lv.assign(n + 1, 0);
    // (một đỉnh của mảnh, tầng)
    vector<pair<int, int>> st = {{1, 0}};
    vector<int> q;
    vector<int> d;
    while (!st.empty()) {
        int r = st.back().first, t = st.back().second;
        st.pop_back();
        collect(r);
        int c = centroid();
        lv[c] = t;
        // BFS từ c trong mảnh, ghi khoảng cách
        q.assign(1, c); d.assign(1, 0); par[c] = 0;
        for (size_t i = 0; i < q.size(); i++) {
            int v = q[i];
            ac[(size_t)v * LG + t] = c; ad[(size_t)v * LG + t] = d[i];
            for (int k = beg[v]; k < beg[v + 1]; k++) {
                int y = to[k];
                if (!removed[y] && y != par[v]) { par[y] = v; q.push_back(y);
d.push_back(d[i] + 1); }
            }
        }
        removed[c] = 1;
        for (int k = beg[c]; k < beg[c + 1]; k++) if (!removed[to[k]])
st.push_back({to[k], t + 1});
    }
}
```

Lê Thành Nam

KHUNG D. Như KHUNG C, khoảng cách kiểu long long

Dùng với KHUNG B: khoảng cách cộng thêm wt[k] thay cho 1.

KHUNG D

```
// số tầng tối đa của cây trọng tâm (2^18 > 2·10^5)
const int LG = 18;
// ac[v * LG + t]: trọng tâm tổ tiên tầng t của v; lv[v]: tầng của v
vector<int> ac, lv;
// ad[v * LG + t]: khoảng cách từ v tới tổ tiên đó
vector<long long> ad;
// trọng tâm tổ tiên tầng t của v
int anc(int v, int t) { return ac[(size_t)v * LG + t]; }
// khoảng cách từ v tới nó
long long dis(int v, int t) { return ad[(size_t)v * LG + t]; }
void buildAnc() {
    ac.assign((size_t)(n + 1) * LG, 0); ad.assign((size_t)(n + 1) * LG, 0);
lv.assign(n + 1, 0);
    // (một đỉnh của mảnh, tầng)
    vector<pair<int, int>> st = {{1, 0}};
    vector<int> q;
    vector<long long> d;
    while (!st.empty()) {
        int r = st.back().first, t = st.back().second;
        st.pop_back();
        collect(r);
        int c = centroid();
        lv[c] = t;
        // BFS từ c trong mảnh, ghi khoảng cách
        q.assign(1, c); d.assign(1, 0); par[c] = 0;
        for (size_t i = 0; i < q.size(); i++) {
            int v = q[i];
            ac[(size_t)v * LG + t] = c; ad[(size_t)v * LG + t] = d[i];
            for (int k = beg[v]; k < beg[v + 1]; k++) {
                int y = to[k];
                if (!removed[y] && y != par[v]) { par[y] = v; q.push_back(y);
d.push_back(d[i] + wt[k]); }
            }
        }
        removed[c] = 1;
        for (int k = beg[c]; k < beg[c + 1]; k++) if (!removed[to[k]])
st.push_back({to[k], t + 1});
    }
}
```

KHUNG E. FFT và tích chập

Biến đổi Fourier nhanh trên số phức như Bài 14 (Tập 1). `conv(f, h)` trả về tích chập của hai mảng đếm; khi tích độ dài hai mảng không quá 4096 thì nhân trực tiếp nhanh hơn. Kết quả là số nguyên tới cỡ n^2, làm tròn bằng `llround`.

Lê Thành Nam

KHUNG E: fft, conv

```cpp
void fft(vector<cd>& a, bool inv) {
    int m = a.size();
    for (int i = 1, j = 0; i < m; i++) {            // hoán vị đảo bit
        int bit = m >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= m; len <<= 1) {
        double ang = 2 * M_PI / len * (inv ? -1 : 1);
        cd wl(cos(ang), sin(ang));
        for (int i = 0; i < m; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i + j], v = a[i + j + len / 2] * w;
                a[i + j] = u + v; a[i + j + len / 2] = u - v;
                w *= wl;
            }
        }
    }
    if (inv) for (auto& x : a) x /= m;
}
// Tích chập hai mảng đếm: res[d] = Σ f[i] · h[d − i]
vector<ll> conv(const vector<ll>& f, const vector<ll>& h) {
    int A = f.size(), B = h.size();
    vector<ll> res(A + B - 1, 0);
    // mảng ngắn: nhân trực tiếp nhanh hơn
    if ((ll)A * B <= 4096) {
        for (int i = 0; i < A; i++) if (f[i]) for (int j = 0; j < B; j++)
res[i + j] += f[i] * h[j];
        return res;
    }
    int m = 1;
    while (m < A + B) m <<= 1;
    vector<cd> a(m), b(m);
    for (int i = 0; i < A; i++) a[i] = cd(f[i], 0);
    for (int i = 0; i < B; i++) b[i] = cd(h[i], 0);
    fft(a, false); fft(b, false);
    for (int i = 0; i < m; i++) a[i] *= b[i];
    fft(a, true);
    for (int i = 0; i < A + B - 1; i++) res[i] = llround(a[i].real());
    return res;
}
```

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU 2
CẤU TRÚC MỘT LỜI GIẢI 3
KHUNG CHƯƠNG TRÌNH 3
BẢNG TỔNG HỢP ĐIỂM THỰC ĐO 13
CHƯƠNG 1. TRỌNG TÂM CỦA CÂY 16
Bài 1.1. CẮT CÂY 17
Bài 1.2. CẮT CẠNH 23
Bài 1.3. TRỌNG TÂM CÂY CON 28
Bài 2.1. CHỢ PHIÊN 34
Bài 2.2. HAI KHO 40
Bài 2.3. CHỢ XÃ 45
Bài 3.1. LỚP CÂY CON 51
Bài 3.2. CÂY ĐỐI XỨNG 56
Bài 3.3. THÊM MỘT LÁ 62
CHƯƠNG 2. PHÂN RÃ TRỌNG TÂM VÀ ĐẾM ĐƯỜNG ĐI 68
Bài 4.1. ĐỘ SÂU CÂY TRỌNG TÂM 70
Bài 4.2. CHỈ HUY 76
Bài 4.3. CÂY LỚN DẦN 83
Bài 5.1. ĐỘ DÀI TỪ L ĐẾN R 90
Bài 5.2. CÙNG MÀU, ĐỨNG K 98
Bài 5.3. ĐỘ DÀI CHIA HẾT 107
Bài 6.1. CẶP GẦN NHAU 114
Bài 6.2. KHOẢNG CÁCH THỨ K 119
Bài 6.3. HAI ĐIỀU KIỆN 125
Bài 7.1. ÍT ĐỈNH ĐẶC BIỆT 134
Bài 7.2. ĐƯỜNG ĐUA DÀI 140
Bài 7.3. NHIỀU ĐÍCH 148
CHƯƠNG 3. CÂY TRỌNG TÂM VÀ TRUY VẤN 155
Bài 8.1. TÔ VÀ XÓA 156
Bài 8.2. ĐỎ GẦN XANH 163
Bài 8.3. ĐỎ XA NHẤT 168
Bài 9.1. TRỌNG SỐ ĐỈNH 174
Bài 9.2. KHO TỐT NHẤT ĐỘNG 179
Bài 9.3. TỔNG BÌNH PHƯƠNG 185
Bài 10.1. VÀNH KHUYÊN 190
Bài 10.2. CẠNH CÓ TRỌNG SỐ 195
Bài 10.3. BẬT TẮT ĐỈNH 200
CHƯƠNG 4. KĨ THUẬT KẾT HỢP CÁC NHÁNH 205
Bài 11.1. XOR BẰNG KHÔNG 206
Bài 11.2. TỔNG CHIA HẾT 214
Bài 11.3. ĐƯỜNG NGOẶC ĐÚNG 223
Bài 12.1. ÍT NHẤT K 231
Bài 12.2. ĐỨNG K ĐỈNH 239
Bài 12.3. TRUNG BÌNH LỚN NHẤT 245
Bài 13.1. NẶNG NHẤT TRONG GIỚI HẠN 254
Bài 13.2. ĐƯỜNG KÍNH QUA TRỌNG TÂM 260
Bài 13.3. ĐẾM ĐƯỜNG DÀI NHẤT 267
CHƯƠNG 5. PHÂN RÃ TRỌNG TÂM NÂNG CAO 275
Bài 14.1. KHOẢNG CÁCH NGUYÊN TỐ 276
Bài 14.2. TỔNG LŨY THỪA 281
Bài 14.3. PHÂN BỐ ĐỎ – XANH 290
Bài 15.1. TỔNG GẦN NHẤT 295
Bài 15.2. ĐOẠN LIÊN TIẾP TỐT NHẤT 301
Bài 15.3. ĐẾM CÁCH CHỌN 308

Lê Thành Nam

BẢNG TỔNG HỢP ĐIỂM THỰC ĐO

Bảng dưới đây cho thấy khoảng cách giữa các cách làm trên toàn bộ 45 bài. Cột "Trung gian" chỉ có ở những bài mà một lời giải nửa vời (đúng một phần, hoặc đúng nhưng chậm) đáng được phân tích riêng.

Bài	Tên bài	Mã	Mức độ	Ngây thơ	Trung gian	Tối ưu
1.1	CẮT CÂY	CATCAY	CƠ BẢN	40	—	**100**
1.2	CẮT CẠNH	CANHCAT	CƠ BẢN	40	—	**100**
1.3	TRỌNG TÂM CÂY CON	TRONGTAMCON	TRUNG BÌNH	30	—	**100**
2.1	CHỢ PHIÊN	CHOPHIEN	CƠ BẢN	30	—	**100**
2.2	HAI KHO	HAIKHO	TRUNG BÌNH	60	—	**100**
2.3	CHỢ XÃ	CHOXA	NÂNG CAO	20	—	**100**
3.1	LỚP CÂY CON	LOPCAY	CƠ BẢN	30	—	**100**
3.2	CÂY ĐỐI XỨNG	DOIXUNG	TRUNG BÌNH	30	—	**100**
3.3	THÊM MỘT LÁ	THEMLA	NÂNG CAO	30	—	**100**
4.1	ĐỘ SÂU CÂY TRỌNG TÂM	DOSAU	CƠ BẢN	30	—	**100**
4.2	CHỈ HUY	CHIAMANH	TRUNG BÌNH	30	50	**100**
4.3	CÂY LỚN DẦN	CAYLON	NÂNG CAO	30	50	**100**
5.1	ĐỘ DÀI TỪ L ĐẾN R	DUONGLR	CƠ BẢN	30	50	**100**
5.2	CÙNG MÀU, ĐÚNG K	DUONGMAU	TRUNG BÌNH	30	60	**100**
5.3	ĐỘ DÀI CHIA HẾT	DUONGCHAN	NÂNG CAO	20	50	**100**
6.1	CẶP GẦN NHAU	CAPGAN	CƠ BẢN	30	—	**100**
6.2	KHOẢNG CÁCH THỨ K	THUK	TRUNG BÌNH	30	—	**100**
6.3	HAI ĐIỀU KIỆN	HAIDK	NÂNG CAO	30	20	**100**
7.1	ÍT ĐỈNH ĐẶC BIỆT	DUAXE2	CƠ BẢN	30	—	**100**
7.2	ĐƯỜNG ĐUA DÀI	DUAXE3	TRUNG BÌNH	30	50	**100**
7.3	NHIỀU ĐÍCH	DUAXEQ	NÂNG CAO	30	50	**100**
8.1	TÔ VÀ XÓA	TOXOA	CƠ BẢN	30	20	**100**

Lê Thành Nam

Bài	Tên bài	Mã	Mức độ	Ngây thơ	Trung gian	Tối ưu
8.2	ĐỎ GẦN XANH	HAIMAU	TRUNG BÌNH	30	—	**100**
8.3	ĐỎ XA NHẤT	XANHAT	NÂNG CAO	30	0	**100**
9.1	TRỌNG SỐ ĐỈNH	TONGTS	CƠ BẢN	30	—	**100**
9.2	KHO TỐT NHẤT ĐỘNG	KHODONG	TRUNG BÌNH	30	—	**100**
9.3	TỔNG BÌNH PHƯƠNG	TONGBP	NÂNG CAO	30	—	**100**
10.1	VÀNH KHUYÊN	VANHKHUYEN	CƠ BẢN	30	—	**100**
10.2	CẠNH CÓ TRỌNG SỐ	DEMGANW	TRUNG BÌNH	30	—	**100**
10.3	BẬT TẮT ĐỈNH	DEMBAT	NÂNG CAO	30	—	**100**
11.1	XOR BẰNG KHÔNG	XORKHONG	CƠ BẢN	30	100	**100**
11.2	TỔNG CHIA HẾT	CHIAHET	TRUNG BÌNH	30	20	**100**
11.3	ĐƯỜNG NGOẶC ĐÚNG	NGOACDUNG	NÂNG CAO	30	50	**100**
12.1	ÍT NHẤT K	ITNHATK	CƠ BẢN	30	20	**100**
12.2	ĐÚNG K ĐỈNH	DUNGK	TRUNG BÌNH	30	20	**100**
12.3	TRUNG BÌNH LỚN NHẤT	TRUNGBINH	NÂNG CAO	30	20	**100**
13.1	NẶNG NHẤT TRONG GIỚI HẠN	NANGNHAT	CƠ BẢN	30	20	**100**
13.2	ĐƯỜNG KÍNH QUA TRỌNG TÂM	DUONGKINH	TRUNG BÌNH	30	20	**100**
13.3	ĐẾM ĐƯỜNG DÀI NHẤT	DEMDAI	NÂNG CAO	30	—	**100**
14.1	KHOẢNG CÁCH NGUYÊN TỐ	NGUYENTO	CƠ BẢN	30	—	**100**
14.2	TỔNG LŨY THỪA	LUYTHUA	TRUNG BÌNH	20	30	**100**
14.3	PHÂN BỐ ĐỎ – XANH	DOXANH	NÂNG CAO	30	—	**100**
15.1	TỔNG GẦN NHẤT	TONGMAX	CƠ BẢN	30	—	**100**
15.2	ĐOẠN LIÊN TIẾP TỐT NHẤT	DOANMAX	TRUNG BÌNH	30	—	**100**
15.3	ĐẾM CÁCH CHỌN	DEMCACH	NÂNG CAO	30	—	**100**

Trung bình, cách ngây thơ đạt 30 điểm, lời giải tối ưu đạt 100 điểm. Phần chênh lệch đó chính là nội dung của cuốn sách này.

Lê Thành Nam

CHƯƠNG 1. TRỌNG TÂM CỦA CÂY

Chín lời giải của chương này chưa cần phân rã: chỉ cần kích thước cây con, kĩ thuật đổi gốc và mã chuẩn AHU. Bài học chung là không tính lại từ đầu: kế thừa trọng tâm của con nặng nhất, dời chợ qua một cạnh, thay xâu mã bằng số hiệu. Cách ngây thơ của các bài này đúng, nhưng sụp đổ trên cây đường thẳng.

Mỗi lời giải đi theo cùng một trình tự bảy phần: hiểu bài toán, cách ngây thơ, bế tắc, phá bế tắc, cách tối ưu, bẫy thường gặp và bảng điểm. Mọi con số trong mục Bế tắc và Bảng điểm là kết quả chấm thật trên bộ test của sách.

TRONG CHƯƠNG NÀY

Bài 1. Trọng tâm của cây: Định nghĩa, sự tồn tại và cách tìm: 1.1 CẮT CÂY · 1.2 CẮT CẠNH · 1.3 TRỌNG TÂM CÂY CON

Bài 2. Trọng tâm có trọng số: Tổng khoảng cách nhỏ nhất: 2.1 CHỢ PHIÊN · 2.2 HAI KHO · 2.3 CHỢ XÃ

Bài 3. Trọng tâm làm gốc chuẩn: Kiểm tra hai cây đồng dạng: 3.1 LỚP CÂY CON · 3.2 CÂY ĐỐI XỨNG · 3.3 THÊM MỘT LÁ

Lê Thành Nam

Điểm thực đo của các cách làm trong chương

Bài	Tên bài	Mức độ	Ngây thơ	Trung gian	Tối ưu
1.1	CẮT CÂY	★	40	—	100
1.2	CẮT CẠNH	★	40	—	100
1.3	TRỌNG TÂM CÂY CON	★★	30	—	100
2.1	CHỢ PHIÊN	★	30	—	100
2.2	HAI KHO	★★	60	—	100
2.3	CHỢ XÃ	★★★	20	—	100
3.1	LỚP CÂY CON	★	30	—	100
3.2	CÂY ĐỐI XỨNG	★★	30	—	100
3.3	THÊM MỘT LÁ	★★★	30	—	100

BÀI 1.1 CẮT CÂY [CATCAY]

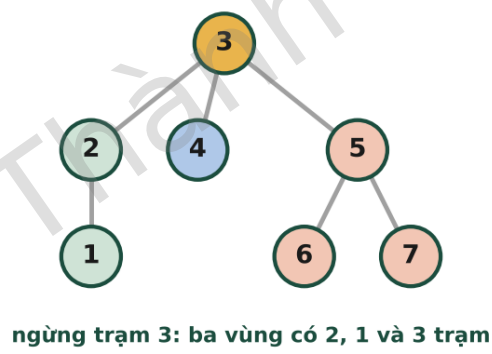
Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
CATCAY.INP	CATCAY.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Bài 1 (Tập 1) · Đề bài: Bài 1.1 (Tập 2)

1. Hiểu bài toán

Ngừng trạm x là xóa đỉnh x khỏi cây: cây vỡ thành các thành phần, mỗi láng giềng của x mở ra một thành phần. Gọi big(x) là số đỉnh của thành phần lớn nhất. Đề hỏi giá trị nhỏ nhất của big(x) và mọi đỉnh đạt giá trị đó.

Đây chính là định nghĩa của **trọng tâm** ở Bài 1 (Tập 1): trọng tâm là đỉnh có big(x) ≤ ⌊n/2⌋, và cây nào cũng có một hoặc hai trọng tâm. Vì trọng tâm luôn tồn tại nên min big(x) ≤ ⌊n/2⌋; mọi đỉnh đạt min đều có big ≤ ⌊n/2⌋, tức là trọng tâm. Ngược lại, khi có hai trọng tâm thì cả hai đều có big = n/2. Vậy tập đỉnh cần in đúng bằng tập trọng tâm.



Hình 1.1.1. *Ví dụ 1: ngừng trạm 3, ba vùng còn lại có 2, 1 và 3 trạm*

2. Cách ngây thơ: Xóa thử từng đỉnh

Với mỗi x, đánh dấu x là đã xóa rồi BFS từ từng láng giềng của x để đếm kích thước các thành phần. Mảng `seen` dùng giá trị "tem" bằng chính x, nên không phải xóa mảng sau mỗi lần thử.

Lê Thành Nam

Cách ngây thơ: O(n²)

```cpp
#include <cstdio>
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("CATCAY.INP", "r")) {
        freopen("CATCAY.INP", "r", stdin);
        freopen("CATCAY.OUT", "w", stdout);
    }
    int n;
    cin >> n;
    vector<vector<int>> g(n + 1);
    for (int i = 0; i < n - 1; i++) {
        int u, v;
        cin >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<int> big(n + 1, 0), seen(n + 1, 0), q;
    // thử ngừng từng trạm x
    for (int x = 1; x <= n; x++) {
        seen[x] = x;
        // mỗi láng giềng của x mở ra một vùng
        for (int s : g[x]) {
            q.assign(1, s);
            seen[s] = x;
            for (size_t i = 0; i < q.size(); i++)
                for (int y : g[q[i]]) if (seen[y] != x) { seen[y] = x;
q.push_back(y); }
            big[x] = max(big[x], (int)q.size());
        }
    }
    int M = *min_element(big.begin() + 1, big.end());
    vector<int> res;
    for (int v = 1; v <= n; v++) if (big[v] == M) res.push_back(v);
    cout << M << ' ' << res.size() << '\n';
    for (size_t i = 0; i < res.size(); i++) cout << res[i] << (i + 1 <
res.size() ? ' ' : '\n');
    return 0;
}
```

Độ phức tạp: Mỗi lần thử duyệt cả cây, tổng $O(n^2)$: $4 \cdot 10^6$ khi $n = 2000$, nhưng $4 \cdot 10^{10}$ khi $n = 2 \cdot 10^5$.

3. Bế tắc

Chấm **cách ngây thơ** trên bộ test của sách:

- Subtask 1 ($n \le 2000$): đúng mọi test, chậm nhất 0,07 giây: 40 điểm.
- Subtask 2 (Không có ràng buộc gì thêm): $n = 2 \cdot 10^5$ cần $4 \cdot 10^{10}$ bước BFS (vượt quá thời gian 1 giây ở 7/7 test): 0 điểm.

> **ĐIỂM BẾ TẮC**
>
> Ta đang tính lại kích thước các thành phần từ đầu cho mỗi x. Nhưng khi xóa x, các thành phần chỉ là **các cây con của x** và **phần còn lại phía trên x**. Nếu biết trước kích thước mọi cây con thì big(x) tính được thế nào?

Le Thanh Nam

4. Phá bế tắc: Một lần BFS tính kích thước cây con

Bước 1 Thứ tự BFS

Đặt gốc tại 1 và BFS để có thứ tự `order` (cha luôn đứng trước con) cùng mảng `par`. Duyệt `order` từ cuối lên, cộng sz[v] vào sz[par[v]]: khi xử lí v thì mọi con của v đã cộng xong vào sz[v]. Không dùng đệ quy, nên cây đường thẳng $2 \cdot 10^5$ đỉnh không làm tràn ngăn xếp.

Bước 2 Tính big(x)

Khi xóa x, các thành phần là cây con của từng con c (kích thước sz[c]) và phần phía trên có n − sz[x] đỉnh (bằng 0 khi x là gốc). Vậy big(x) = max(n − sz[x], max sz[c]). Tổng số lần xét cạnh là O(n).

Bước 3 Chọn kết quả

Lấy M = min big(x), rồi in mọi x có big(x) = M theo thứ tự tăng dần (duyệt x từ 1 tới n là đủ).

Chạy tay ví dụ 1 (gốc tại 1, n = 7)

Đỉnh x	sz[x]	n − sz[x]	Cây con lớn nhất	big(x)
1	7	0	6 (đỉnh 2)	6
2	6	1	5 (đỉnh 3)	5
3	5	2	3 (đỉnh 5)	**3**
4	1	6	—	6
5	3	4	1	4
6, 7	1	6	—	6

M = 3 và chỉ có đỉnh 3 đạt: kết quả "3 1" rồi "3".

> **VÌ SAO ĐÚNG?**
>
> Khi xóa x, mỗi con c của x cùng toàn bộ cây con của nó tạo thành một thành phần sz[c] đỉnh; các đỉnh không thuộc cây con của x tạo thành đúng một thành phần (liên thông qua cha của x) có n − sz[x] đỉnh. Vậy công thức big(x) là chính xác.
>
> Trọng tâm luôn tồn tại (Bài 1, Tập 1) nên M ≤ ⌊n/2⌋; đỉnh đạt M chính là các trọng tâm.

Lê Thành Nam

5. Cách tối ưu

CATCAY.CPP: Kích thước cây con, O(n)

```cpp
#include <cstdio>
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("CATCAY.INP", "r")) {
        freopen("CATCAY.INP", "r", stdin);
        freopen("CATCAY.OUT", "w", stdout);
    }
    int n;
    cin >> n;
    vector<vector<int>> g(n + 1);
    for (int i = 0; i < n - 1; i++) {
        int u, v;
        cin >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<int> order = {1}, par(n + 1, 0), sz(n + 1, 1);
    par[1] = -1;
    // thứ tự BFS: cha đứng trước con
    for (size_t i = 0; i < order.size(); i++) {
        int v = order[i];
        for (int c : g[v]) if (c != par[v]) { par[c] = v; order.push_back(c);
}
    }
    for (int i = n - 1; i > 0; i--) sz[par[order[i]]] += sz[order[i]];
    vector<int> big(n + 1);
    // vùng phía cha và các cây con
    for (int v = 1; v <= n; v++) {
        big[v] = n - sz[v];
        for (int c : g[v]) if (c != par[v]) big[v] = max(big[v], sz[c]);
    }
    int M = *min_element(big.begin() + 1, big.end());
    vector<int> res;
    for (int v = 1; v <= n; v++) if (big[v] == M) res.push_back(v);
    cout << M << ' ' << res.size() << '\n';
    for (size_t i = 0; i < res.size(); i++) cout << res[i] << (i + 1 <
res.size() ? ' ' : '\n');
    return 0;
}
```

Lê Thành Nam

Độ phức tạp: Một lần BFS và một lần duyệt mọi cạnh: O(n).

> **MỞ RỘNG: CÂY CÓ TRỌNG SỐ ĐỈNH**
> Nếu mỗi trạm có "tải" a_x và cần cực tiểu tổng tải của vùng lớn nhất, chỉ cần thay sz bằng tổng trọng số cây con. Đó chính là trọng tâm có trọng số của Bài 2.

6. Bẫy thường gặp

Quên phần phía trên n − sz[x]: khi đó gốc của BFS luôn trông như tốt nhất.

Chỉ in một trọng tâm: đường thẳng có số đỉnh chẵn có hai trọng tâm, cả hai đều phải in.

Duyệt cây bằng đệ quy: đường thẳng $2 \cdot 10^5$ đỉnh làm tràn ngăn xếp 1 MB của Themis.

7. Bảng điểm theo thuật toán

Thuật toán	Subtask 1 (40)	Subtask 2 (60)	Tổng
Xóa thử từng đỉnh, $O(n^2)$	40	0	**40**
Kích thước cây con, $O(n)$	40	60	**100**

Lê Thành Nam